МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
Е.И. Мицкевич
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования и паттернов проектирования на языке Python

Ход работы

Задание 1 (порождающий паттерн строитель)

4) Проект «Туристическое бюро». Реализовать возможность выбора программы тура (проезд, проживание, питание, посещение музеев, выставок, экскурсии и т.д.). Должна формироваться итоговая стоимость заказа.

```
fromabcimportABC, abstractmethod
fromenumimportEnum
fromtypingimportList

classTransportType(Enum):
    """Типы транспорта для проезда"""
    ECONOMY="Эконом (автобус)"
    STANDARD="Стандарт (поезд)"
    BUSINESS="Бизнес (самолет)"
    LUXURY="Люкс (индивидуальный трансфер)"

classAccommodationType(Enum):
    """Типы проживания"""
    HOSTEL="Хостел"
    HOTEL_3_STARS="Отель 3 звезды"
    HOTEL_4_STARS="Отель 4 звезды"
    HOTEL_5_STARS="Отель 5 звезд"
    APARTMENTS="Апартаменты"


classMealType(Enum):
    """Типы питания"""
    WITHOUT_MEALS="Без питания"
    BREAKFAST_ONLY="Только завтраки"
    HALF_BOARD="Полупансион (завтрак + ужин)"
    FULL_BOARD="Полный пансион (3-разовое)"
    ALL_INCLUSIVE="Все включено"


classTourPackage:
    """
    Класс туристического пакета (продукт)
    """


    def__init__(self):
        self._transport=None
        self._accommodation=None
        self._meal=None
        self._excursions: List[str] =[]
        self._museums: List[str] =[]
        self._additional_services: List[str] =[]


    defset_transport(self, transport: TransportType, price: float):
        self._transport={
            'type': transport,
            'price': price
        }


    defset_accommodation(self, accommodation: AccommodationType, nights: int,
price_per_night: float):
        self._accommodation={
            'type': accommodation,
            'nights': nights,
```

```
            'price_per_night': price_per_night
        }


    defset_meal(self, meal: MealType, price: float):
        self._meal={
            'type': meal,
            'price': price
        }


    defadd_excursion(self, name: str, price: float):
        self._excursions.append({
            'name': name,
            'price': price
        })

    defadd_museum(self, name: str, price: float):
        self._museums.append({
            'name': name,
            'price': price
        })

    defadd_service(self, name: str, price: float):
        self._additional_services.append({
            'name': name,
            'price': price
        })


    defcalculate_total_cost(self) -> float:
        """Расчет общей стоимости тура"""
        total=0

        # Проезд
        ifself._transport:
            total+=self._transport['price']

        # Проживание
        ifself._accommodation:
            total+=self._accommodation['nights']
*self._accommodation['price_per_night']


        # Питание
        ifself._meal:
            total+=self._meal['price']


        # Экскурсии
        forexcursioninself._excursions:
            total+=excursion['price']

        # Музеи
        formuseuminself._museums:
            total+=museum['price']


        # Доп. услуги
        forserviceinself._additional_services:
            total+=service['price']

        returntotal

    def__str__(self) -> str:
        """Строковое представление тура"""
        result=["*"*50, "ВАШ ТУРИСТИЧЕСКИЙ ПАКЕТ", "*"*50]


        # Проезд
```

```
        ifself._transport:
            result.append(f"\nПроезд: {self._transport['type'].value}")
            result.append(f"   Стоимость: {self._transport['price']}$")

        # Проживание
        ifself._accommodation:
            result.append(f"\nПроживание:
{self._accommodation['type'].value}")
            result.append(f"   Количество ночей:
{self._accommodation['nights']}")
            result.append(f"   Стоимость за ночь:
{self._accommodation['price_per_night']}$")
            result.append(f"   Всего: {self._accommodation['nights']
*self._accommodation['price_per_night']}$")

        # Питание
        ifself._meal:
            result.append(f"\nПитание: {self._meal['type'].value}")
            result.append(f"   Стоимость: {self._meal['price']}$")

        # Экскурсии
        ifself._excursions:
            result.append("\nЭкскурсии:")
            foreinself._excursions:
                result.append(f"   • {e['name']}- {e['price']}$")

        # Музеи
        ifself._museums:
            result.append("\nПосещение музеев:")
            forminself._museums:
                result.append(f"  {m['name']}- {m['price']}$")


        # Доп. услуги
        ifself._additional_services:
            result.append("\nДополнительные услуги:")
            forsinself._additional_services:
                result.append(f"   • {s['name']}- {s['price']}$")


        result.append(f"\nИТОГОВАЯ СТОИМОСТЬ:
{self.calculate_total_cost()}$")
        result.append("*"*50)

        return"\n".join(result)

classTourBuilder(ABC):
    """
    Абстрактный строитель туров
    """

    def__init__(self):
        self._tour=None

    defcreate_new_tour(self):
        """Создание нового тура"""
        self._tour=TourPackage()

    @abstractmethod
    defbuild_transport(self):
        pass


    @abstractmethod
    defbuild_accommodation(self):
        pass


    @abstractmethod
    defbuild_meal(self):
```

```
        pass

    @abstractmethod
    defbuild_excursions(self):
        pass


    @abstractmethod
    defbuild_museums(self):
        pass


    @abstractmethod
    defbuild_additional_services(self):
        pass

    defget_tour(self) -> TourPackage:
        """Получение готового тура"""
        returnself._tour


classEconomyTourBuilder(TourBuilder):
    """
    Строитель эконом-тура
    """

    defbuild_transport(self):
        self._tour.set_transport(TransportType.ECONOMY, 300)


    defbuild_accommodation(self):
        self._tour.set_accommodation(AccommodationType.HOSTEL, 5, 100)


    defbuild_meal(self):
        self._tour.set_meal(MealType.WITHOUT_MEALS, 0)


    defbuild_excursions(self):
        self._tour.add_excursion("Обзорная экскурсия по городу", 50)


    defbuild_museums(self):
        self._tour.add_museum("Краеведческий музей", 20)


    defbuild_additional_services(self):
        pass # Нет доп. услуг в эконом-туре


classStandardTourBuilder(TourBuilder):
    """
    Строитель стандартного тура
    """

    defbuild_transport(self):
        self._tour.set_transport(TransportType.STANDARD, 1000)


    defbuild_accommodation(self):
        self._tour.set_accommodation(AccommodationType.HOTEL_3_STARS, 5, 250)


    defbuild_meal(self):
        self._tour.set_meal(MealType.BREAKFAST_ONLY, 200)


    defbuild_excursions(self):
        self._tour.add_excursion("Обзорная экскурсия по городу", 50)
        self._tour.add_excursion("Посещение центра с Эльфилевой башней", 80)
```

```
    defbuild_museums(self):
        self._tour.add_museum("Краеведческий музей", 20)
        self._tour.add_museum("Лувр (стандарт)", 30)


    defbuild_additional_services(self):
        self._tour.add_service("Встреча в аэропорту", 50)
        self._tour.add_service("Медицинская страховка", 50)


classLuxuryTourBuilder(TourBuilder):
    """
    Строитель люкс-тура
    """

    defbuild_transport(self):
        self._tour.set_transport(TransportType.LUXURY, 1500)


    defbuild_accommodation(self):
        self._tour.set_accommodation(AccommodationType.HOTEL_5_STARS, 7,
1000)


    defbuild_meal(self):
        self._tour.set_meal(MealType.ALL_INCLUSIVE, 1000)


    defbuild_excursions(self):
        self._tour.add_excursion("Индивидуальная экскурсия по городу", 500)
        self._tour.add_excursion("Винный тур с дегустацией", 300)
        self._tour.add_excursion("Морская прогулка на яхте", 1000)


    defbuild_museums(self):
        self._tour.add_museum("Лувр (VIP-билеты)", 300)
        self._tour.add_museum("Обзорная площадка на Эльфелевой башне", 200)


    defbuild_additional_services(self):
        self._tour.add_service("Индивидуальный гид", 800)
        self._tour.add_service("Спа-процедуры", 500)


classCustomTourBuilder(TourBuilder):
    """
    Строитель для кастомного тура (с возможностью выбора)
    """


    def__init__(self):
        super().__init__()
        self._transport_choice=None
        self._accommodation_choice=None
        self._meal_choice=None
        self._nights=5
        self._selected_excursions=[]
        self._selected_museums=[]
        self._selected_services=[]


    defset_transport_choice(self, transport: TransportType, price: float):
        self._transport_choice=(transport, price)


    defset_accommodation_choice(self, accommodation: AccommodationType,
                                nights: int, price_per_night: float):
        self._accommodation_choice=(accommodation, nights, price_per_night)
```

```
    defset_meal_choice(self, meal: MealType, price: float):
        self._meal_choice=(meal, price)


    defadd_excursion_choice(self, name: str, price: float):
        self._selected_excursions.append((name, price))


    defadd_museum_choice(self, name: str, price: float):
        self._selected_museums.append((name, price))


    defadd_service_choice(self, name: str, price: float):
        self._selected_services.append((name, price))


    defbuild_transport(self):
        ifself._transport_choice:
            transport, price=self._transport_choice
            self._tour.set_transport(transport, price)


    defbuild_accommodation(self):
        ifself._accommodation_choice:
            accommodation, nights, price_per_night=self._accommodation_choice
            self._tour.set_accommodation(accommodation, nights,
price_per_night)


    defbuild_meal(self):
        ifself._meal_choice:
            meal, price=self._meal_choice
            self._tour.set_meal(meal, price)


    defbuild_excursions(self):
        forname, priceinself._selected_excursions:
            self._tour.add_excursion(name, price)


    defbuild_museums(self):
        forname, priceinself._selected_museums:
            self._tour.add_museum(name, price)


    defbuild_additional_services(self):
        forname, priceinself._selected_services:
            self._tour.add_service(name, price)

classTourDirector:
    """
    Директор, управляющий процессом строительства
    """

    def__init__(self, builder: TourBuilder):
        self._builder=builder


    defconstruct_economy_tour(self):
        """Построение эконом-тура"""
        self._builder.create_new_tour()
        self._builder.build_transport()
        self._builder.build_accommodation()
        self._builder.build_meal()
        self._builder.build_excursions()
        self._builder.build_museums()
        self._builder.build_additional_services()
        returnself._builder.get_tour()
```

```
    defconstruct_standard_tour(self):
        """Построение стандартного тура"""
        self._builder.create_new_tour()
        self._builder.build_transport()
        self._builder.build_accommodation()
        self._builder.build_meal()
        self._builder.build_excursions()
        self._builder.build_museums()
        self._builder.build_additional_services()
        returnself._builder.get_tour()


    defconstruct_luxury_tour(self):
        """Построение люкс-тура"""
        self._builder.create_new_tour()
        self._builder.build_transport()
        self._builder.build_accommodation()
        self._builder.build_meal()
        self._builder.build_excursions()
        self._builder.build_museums()
        self._builder.build_additional_services()
        returnself._builder.get_tour()


    defconstruct_custom_tour(self):
        """Построение кастомного тура"""
        self._builder.create_new_tour()
        self._builder.build_transport()
        self._builder.build_accommodation()
        self._builder.build_meal()
        self._builder.build_excursions()
        self._builder.build_museums()
        self._builder.build_additional_services()
        returnself._builder.get_tour()


classTravelAgency:
    """
    Класс туристического агентства (клиент)
    """

    def__init__(self):
        self._available_excursions=[
            ("Обзорная экскурсия по городу", 50),
            ("Исторический центр", 80),
            ("Винный тур с дегустацией", 300),
            ("Морская прогулка", 1000),
            ("Ночная экскурсия", 100)
        ]


        self._available_museums=[
            ("Краеведческий музей", 20),
            ("Художественная галерея", 30),
            ("Лувр", 300),
            ("Обзорная площадка на Эльфелевой башне", 500),
        ]

        self._available_services=[
            ("Встреча в аэропорту", 100),
            ("Медицинская страховка", 50),
            ("Индивидуальный гид", 200),
            ("Спа-процедуры", 300),
            ("Фотосессия", 150)
        ]


    defshow_menu(self):
```

```
        """Показать меню выбора тура"""
        print("\n"+"*"*50)
        print("ДОБРО ПОЖАЛОВАТЬ В ТУРИСТИЧЕСКОЕ БЮРО")
        print("*"*50)
        print("1. Готовый эконом-тур")
        print("2. Готовый стандартный тур")
        print("3. Готовый люкс-тур")
        print("4. Собрать свой тур")
        print("0. Выход")
        print("*"*50)

    defcreate_custom_tour(self):
        """Создание кастомного тура"""
        builder=CustomTourBuilder()
        director=TourDirector(builder)

        print("\n--- СОЗДАНИЕ ИНДИВИДУАЛЬНОГО ТУРА ---")

        # Выбор транспорта
        print("\nВыберите тип транспорта:")
        fori, transportinenumerate(TransportType, 1):
            price={1: 150, 2: 300, 3: 800, 4: 1500}[i]
            print(f"{i}. {transport.value}- {price}$")

        transport_choice=int(input("Ваш выбор (1-4): "))
        transport=list(TransportType)[transport_choice-1]
        transport_price=[150, 300, 800, 1500][transport_choice-1]
        builder.set_transport_choice(transport, transport_price)

        # Выбор проживания
        print("\nВыберите тип проживания:")
        fori, accommodationinenumerate(AccommodationType, 1):
            price={1: 80, 2: 150, 3: 250, 4: 500, 5: 300}[i]
            print(f"{i}. {accommodation.value}- {price}$/ночь")


        accommodation_choice=int(input("Ваш выбор (1-5): "))
        accommodation=list(AccommodationType)[accommodation_choice-1]
        nights=int(input("Количество ночей: "))
        price_per_night=[80, 150, 250, 500, 300][accommodation_choice-1]
        builder.set_accommodation_choice(accommodation, nights,
price_per_night)


        # Выбор питания
        print("\nВыберите тип питания:")
        fori, mealinenumerate(MealType, 1):
            price={1: 0, 2: 100, 3: 250, 4: 400, 5: 600}[i]
            print(f"{i}. {meal.value}- {price}$")


        meal_choice=int(input("Ваш выбор (1-5): "))
        meal=list(MealType)[meal_choice-1]
        meal_price=[0, 100, 250, 400, 600][meal_choice-1]
        builder.set_meal_choice(meal, meal_price)


        # Выбор экскурсий
        print("\nДоступные экскурсии (введите номера через пробел, 0 -
пропустить):")
        fori, (name, price) inenumerate(self._available_excursions, 1):
            print(f"{i}. {name}- {price}$")


        excursions_choice=input("Ваш выбор: ").strip()
        ifexcursions_choice!="0"andexcursions_choice:
            fornuminexcursions_choice.split():
                idx=int(num) -1
                if0<=idx<len(self._available_excursions):
                    name, price=self._available_excursions[idx]
```

```
                    builder.add_excursion_choice(name, price)


        # Выбор музеев
        print("\nДоступные музеи (введите номера через пробел, 0 -
пропустить):")
        fori, (name, price) inenumerate(self._available_museums, 1):
            print(f"{i}. {name}- {price}$")


        museums_choice=input("Ваш выбор: ").strip()
        ifmuseums_choice!="0"andmuseums_choice:
            fornuminmuseums_choice.split():
                idx=int(num) -1
                if0<=idx<len(self._available_museums):
                    name, price=self._available_museums[idx]
                    builder.add_museum_choice(name, price)

        # Выбор доп. услуг
        print("\nДополнительные услуги (введите номера через пробел, 0 -
пропустить):")
        fori, (name, price) inenumerate(self._available_services, 1):
            print(f"{i}. {name}- {price}$")


        services_choice=input("Ваш выбор: ").strip()
        ifservices_choice!="0"andservices_choice:
            fornuminservices_choice.split():
                idx=int(num) -1
                if0<=idx<len(self._available_services):
                    name, price=self._available_services[idx]
                    builder.add_service_choice(name, price)

        # Строим тур
        tour=director.construct_custom_tour()
        returntour

    defrun(self):
        """Запуск работы агентства"""
        whileTrue:
            self.show_menu()
            choice=input("Выберите действие: ")


            ifchoice=="0":
                print("Спасибо за обращение! До свидания!")
                break

            builder=None
            tour=None

            ifchoice=="1":
                builder=EconomyTourBuilder()
                director=TourDirector(builder)
                tour=director.construct_economy_tour()
                print("\nСформирован эконом-тур:")


            elifchoice=="2":
                builder=StandardTourBuilder()
                director=TourDirector(builder)
                tour=director.construct_standard_tour()
                print("\nСформирован стандартный тур:")


            elifchoice=="3":
                builder=LuxuryTourBuilder()
                director=TourDirector(builder)
                tour=director.construct_luxury_tour()
                print("\nСформирован люкс-тур:")
```

```
        elifchoice=="4":
            tour=self.create_custom_tour()
            print("\nСформирован индивидуальный тур:")


        iftour:
            print(tour)


            confirm=input("\nЖелаете оформить тур? (да/нет): ")
            ifconfirm.lower() =="да":
                print("Тур успешно оформлен! Приятного отдыха!")
            else:
                print("Оформление отменено")
```

```
**************************************************
ДОБРО ПОЖАЛОВАТЬ В ТУРИСТИЧЕСКОЕ БЮРО
**************************************************
1. Готовый эконом-тур
2. Готовый стандартный тур
3. Готовый люкс-тур
4. Собрать свой тур
0. Выход
**************************************************
Выберите действие: 1

Сформирован эконом-тур:
**************************************************
ВАШ ТУРИСТИЧЕСКИЙ ПАКЕТ
**************************************************

Проезд: Эконом (автобус)
   Стоимость: 300 $

Проживание: Хостел
   Количество ночей: 5
   Стоимость за ночь: 100 $
   Всего: 500 $

Питание: Без питания
   Стоимость: 0 $

Экскурсии:
   • Обзорная экскурсия по городу - 50 $

Посещение музеев:
  Краеведческий музей - 20 $

ИТОГОВАЯ СТОИМОСТЬ: 870 $
**************************************************
```

```
Сформирован люкс-тур:
**************************************************
ВАШ ТУРИСТИЧЕСКИЙ ПАКЕТ
**************************************************

Проезд: Люкс (индивидуальный трансфер)
   Стоимость: 1500 $

Проживание: Отель 5 звезд
   Количество ночей: 7
   Стоимость за ночь: 1000 $
   Всего: 7000 $

Питание: Все включено
   Стоимость: 1000 $

Экскурсии:
   • Индивидуальная экскурсия по городу - 500 $
   • Винный тур с дегустацией - 300 $
   • Морская прогулка на яхте - 1000 $

Посещение музеев:
  Лувр (VIP-билеты) - 300 $
  Обзорная площадка на Эльфелевой башне - 200 $

Дополнительные услуги:
   • Индивидуальный гид - 800 $
   • Спа-процедуры - 500 $

ИТОГОВАЯ СТОИМОСТЬ: 13100 $
**************************************************

Желаете оформить тур? (да/нет): да
Тур успешно оформлен! Приятного отдыха!
```

```
Сформирован индивидуальный тур:
**************************************************
ВАШ ТУРИСТИЧЕСКИЙ ПАКЕТ
**************************************************

Проезд: Бизнес (самолет)
   Стоимость: 800 $

Проживание: Отель 3 звезды
   Количество ночей: 7
   Стоимость за ночь: 150 $
   Всего: 1050 $

Питание: Полупансион (завтрак + ужин)
   Стоимость: 250 $

Экскурсии:
   • Обзорная экскурсия по городу - 50 $
   • Исторический центр - 80 $
   • Винный тур с дегустацией - 300 $
   • Ночная экскурсия - 100 $

Посещение музеев:
  Лувр - 300 $
  Художественная галерея - 30 $

Дополнительные услуги:
   • Медицинская страховка - 50 $
   • Фотосессия - 150 $

ИТОГОВАЯ СТОИМОСТЬ: 3160 $
**************************************************

Желаете оформить тур? (да/нет):
```

Задание 2 (структурный паттерн компоновщик)

4) Проект «Файловая система». Реализуйте модель работы файловой системы. Должна поддерживаться иерархичность ФС на уровне директорий и отдельных файлов. Файлы могут иметь все основные присущие им атрибуты (размер, расширение, дата создания и т.д.).

```
from abc import ABC, abstractmethod

class FileSystemComponent(ABC):
    """Абстрактный базовый класс для всех компонентов файловой системы"""

    def__init__(self, name, date):
        self._name=name
        self._date=date

    @abstractmethod
    defgetSize(self):
        pass

    @abstractmethod
    defdisplay(self, indent=""):
        pass
```

```
    defgetName(self):
        returnself._name

    defgetDate(self):
        returnself._date

    defadd(self, component):
        raiseNotImplementedError("Сюда добавлять текст низя :( )")

    defremove(self, component):
        raiseNotImplementedError("Удалить низя :( )")

    defgetChild(self, index):
        raiseNotImplementedError("Упс, что-то пошло не так *_* ")



"""Класс папочки"""
classDirectory(FileSystemComponent):

    def__init__(self, name, date):
        super().__init__(name, date)
        self._children=[]

    defadd(self, component):
        """Добавление компонента в директорию"""
        ifcomponentnotinself._children:
            self._children.append(component)
            print(f"{component.getName()}добавлен в папочку {self._name}")
        else:
            print(f"{component.getName()}уже есть в папочке {self._name}")

    defremove(self, component):
        """Удаление компонента из директории"""
        ifcomponentinself._children:
            self._children.remove(component)
            print(f"{component.getName()}удален из папочки {self._name},
дасвиданя ;)")

    defgetChild(self, index):
        """Получение дочернего компонента по индексу"""
        if0<=index<len(self._children):
            returnself._children[index]
        returnNone

    defgetChildren(self):
        """Получение всех дочерних компонентов"""
        returnself._children.copy()

    defgetSize(self):
        """Рекурсивный подсчет размера папки"""
        total_size=0
        forchildinself._children:
            total_size+=child.getSize()
        returntotal_size

    deffindByName(self, name):
        """Поиск компонента по имени в текущей папке"""
        forchildinself._children:
            ifchild.getName() ==name:
                returnchild
        returnNone

    defdisplay(self, indent=""):
        """Вывод содержимого папки"""
        print(f"{indent}{self._name}/ (размер: {self.getSize()}байт, создан:
{self._date})")
        forchildinself._children:
            child.display(indent+"  ")
```

```
"""Класс файл"""
classFile(FileSystemComponent):

    def__init__(self, name, extension, date):
        super().__init__(f"{name}.{extension}", date)
        self._name_without_ext=name
        self._extension=extension
        self._size=0
        self._content=""

    defgetSize(self):
        returnself._size

    defgetExtension(self):
        returnself._extension

    defaddContent(self):
        """Добавление содержимого в файл"""
        ifself._extension.lower() =="txt":
            contOfFile=input("Введите содержимое текстового файла: ")
            self._content=contOfFile
            self._size=len(contOfFile.encode('utf-8'))  # Размер в байтах
            print(f"Содержимое добавлено в файл {self._name}. Размер:
{self._size}байт")
        else:
            print(f"Это не текстовый файл (расширение .{self._extension}),
вводить низя :(")

    defreadContent(self):
        """Чтение содержимого файла"""
        ifself._extension.lower() =="txt"andself._content:
            print(f"\nСодержимое файла {self._name}:")
            print(self._content)
            print()
        elifself._extension.lower() =="txt":
            print(f"Файл {self._name}пуст")
        else:
            print(f"Файл {self._name}- бинарный, прочитать как текст низя :(
)")

    defdisplay(self, indent=""):
        """Отображение информации о файле"""
        print(f"{indent}{self._name}(размер: {self._size}байт, создан:
{self._date})")



"""Класс для управления файловой системой в целом"""
classFileSystem:

    def__init__(self):
        # При создании корня просим пользователя ввести дату
        root_date=input("Введите дату создания корневой директории (например,
2024-01-15): ")
        self._root=Directory("root", root_date)
        self._current_directory=self._root
        self._current_path=["root"]

    defgetCurrentPath(self):
        """Получение текущего пути"""
        return"/"+"/".join(self._current_path)

    defchangeDirectory(self, dirname):
        """Смена текущей директории"""
        ifdirname==".":
            iflen(self._current_path) >1:
                self._current_path.pop()
```

```
            # Перестроение текущей директории от корня
            self._current_directory=self._root
            fordir_nameinself._current_path[1:]:
                next_dir=self._current_directory.findByName(dir_name)
                ifnext_dirandisinstance(next_dir, Directory):
                    self._current_directory=next_dir
                else:
                    break
            print(f"Перешли в директорию {self.getCurrentPath()}")
        else:
            print("Уже в корневой директории")
    else:
        # Поиск поддиректории в текущей
        target=self._current_directory.findByName(dirname)
        iftargetandisinstance(target, Directory):
            self._current_directory=target
            self._current_path.append(dirname)
            print(f"Перешли в папочку {self.getCurrentPath()}")
        else:
            print(f"Папка '{dirname}' не найдена, плак плак ((( )")

defcreateFile(self, name, extension):
    """Создание файла в текущей папке"""
    # Проверка на существование файла с таким именем
    filename=f"{name}.{extension}"
    existing=self._current_directory.findByName(filename)
    ifexisting:
        print(f"Файл '{filename}' уже существует")
        returnNone

    # Просим пользователя ввести дату создания файла
    file_date=input(f"Введите дату создания файла {filename}: ")
    new_file=File(name, extension, file_date)
    self._current_directory.add(new_file)
    returnnew_file

defcreateDirectory(self, name):
    """Создание папки в папке"""
    existing=self._current_directory.findByName(name)
    ifexisting:
        print(f"Папка '{name}' уже существует")
        returnNone

    # Просим пользователя ввести дату создания папки
    dir_date=input(f"Введите дату создания папки {name}: ")
    new_dir=Directory(name, dir_date)
    self._current_directory.add(new_dir)
    returnnew_dir

deflistContents(self):
    """Вывод содержимого текущей папки"""
    print(f"\nСодержимое {self.getCurrentPath()}:")
    ifnotself._current_directory.getChildren():
        print("  (пусто, выросла капуста, хы)")
    else:
        self._current_directory.display()

deffindFile(self, filename):
    """Поиск файла в текущей папке"""
    returnself._current_directory.findByName(filename)

defdeleteItem(self, name):
    """Удаление файла или папки из текущей папки"""
    item=self._current_directory.findByName(name)
    ifitem:
        self._current_directory.remove(item)
    else:
        print(f"'{name}' не найден в текущей папке, какой ужассс")

defgetCurrentDirectory(self):
```

```
        """Получение объекта текущей папки"""
        returnself._current_directory



defMenu():
    print("*"*50)
    print("Команды:")
    print("  ls              - показать содержимое текущей директории")
    print("  cd <директория> - перейти в директорию")
    print("  cd ..           - перейти в родительскую директорию")
    print("  mkdir <имя>     - создать директорию")
    print("  touch <имя>.<расш> - создать файл")
    print("  write <файл>    - добавить содержимое в txt файл")
    print("  read <файл>     - прочитать содержимое txt файла")
    print("  rm <имя>        - удалить файл/директорию")
    print("  pwd             - показать текущий путь")
    print("  tree            - показать всё дерево от корня")
    print("  help            - показать команды")
    print("  exit            - выход")
    print("*"*50)



"""Интерактивный интерфейс для работы с файловой системой"""
print("*"*50)
print("ФАЙЛОВАЯ СИСТЕМА v1.0")
print("*"*50)
Menu()


# Создаем файловую систему с вводом даты для корня
fs=FileSystem()


whileTrue:
    command=input(f"\n{fs.getCurrentPath()}> ").strip().split()

    ifnotcommand:
        continue

    cmd=command[0].lower()

    ifcmd=="exit":
        print("До свидания!")
        break

    elifcmd=="help":
        Menu()

    elifcmd=="ls":
        fs.listContents()

    elifcmd=="pwd":
        print(fs.getCurrentPath())

    elifcmd=="tree":
        print("\nПолное дерево файловой системы:")
        fs._root.display()

    elifcmd=="cd":
        iflen(command) <2:
            print("Укажите папочку")
        else:
            fs.changeDirectory(command[1])

    elifcmd=="mkdir":
        iflen(command) <2:
            print("Укажите имя папочки")
        else:
```

```
            fs.createDirectory(command[1])

    elifcmd=="touch":
        iflen(command) <2:
            print("Укажите имя файла с расширением (например: document.txt)")
        else:
            filename=command[1]
            if"."infilename:
                name, ext=filename.rsplit(".", 1)
                fs.createFile(name, ext)
            else:
                print("Файл должен иметь расширение (например:
document.txt)")

    elifcmd=="write":
        iflen(command) <2:
            print("Укажите имя файла")
        else:
            file_item=fs.findFile(command[1])
            iffile_itemandisinstance(file_item, File):
                file_item.addContent()
            else:
                print(f"Файл '{command[1]}' не найден")

    elifcmd=="read":
        iflen(command) <2:
            print("Укажите имя файла")
        else:
            file_item=fs.findFile(command[1])
            iffile_itemandisinstance(file_item, File):
                file_item.readContent()
            else:
                print(f"Файл '{command[1]}' не найден")

    elifcmd=="rm":
        iflen(command) <2:
            print("Укажите имя файла или директории")
        else:
            fs.deleteItem(command[1])

    else:
        print(f"Неизвестная команда: {cmd}")
```

```
/root> help
==================================================
Команды:
  ls              - показать содержимое текущей директории
  cd <директория> - перейти в директорию
  cd ..           - перейти в родительскую директорию
  mkdir <имя>     - создать директорию
  touch <имя>.<расш> - создать файл
  write <файл>    - добавить содержимое в txt файл
  read <файл>     - прочитать содержимое txt файла
  rm <имя>        - удалить файл/директорию
  pwd             - показать текущий путь
  tree            - показать всё дерево от корня
  help            - показать команды
  exit            - выход
==================================================

/root> mkdir papochka
Введите дату создания папки papochka: 2025-05-04
papochka добавлен в папочку root

/root> touch file.txt
Введите дату создания файла file.txt: 2026-02-02
file.txt добавлен в папочку root
```

```
/root> write file.txt
Введите содержимое текстового файла: nanananananananan ananananananan
Содержимое добавлено в файл file.txt. Размер: 30 байт

/root> tree

Полное дерево файловой системы:
root/ (размер: 30 байт, создан: & C:/Users/Huawei/AppData/Local/Prog
tskevich/3/src/2.4.py)
  papochka/ (размер: 0 байт, создан: 2025-05-04)
   file.txt (размер: 30 байт, создан: 2026-02-02)

/root> read
Укажите имя файла

/root> read file.txt

Содержимое файла file.txt:
nanananananananan ananananananan


/root> rm file.txt
file.txt удален из папочки root, дасвиданя ;)

/root> rm file.txt
'file.txt' не найден в текущей папке, какой ужассс
```

Задание 3 (поведенческий паттерн итератор)

4) Реализовать вывод ФС из 2-й группы заданий. Вывод файлов/директорий должен осуществляться в случайном порядке. Вывести основные атрибуты каждого файла/директории.

```
# ПАТТЕРН ИТЕРАТОР
classComponentIterator(ABC):
    """Абстрактный класс итератора"""


    @abstractmethod
    defhas_next(self) -> bool:
        pass


    @abstractmethod
    defnext(self):
        pass



classRandomIterator(ComponentIterator):
    """Конкретный итератор для обхода компонентов в случайном порядке"""


    def__init__(self, components: List):
        self._components=components.copy()  # Создаем копию списка
        self._index=0
        self._shuffle_components()  # Перемешиваем компоненты при создании
итератора


    def_shuffle_components(self):
        """Перемешивает компоненты в случайном порядке"""
        random.shuffle(self._components)
        print(f"    [Итератор: компоненты перемешаны для случайного вывода]")


    defhas_next(self) -> bool:
        """Проверяет, есть ли еще элементы для обхода"""
        returnself._index<len(self._components)


    defnext(self):
        """Возвращает следующий элемент и переходит к следующему"""
        ifself.has_next():
            component=self._components[self._index]
            self._index+=1
            returncomponent
        else:
            raiseStopIteration("Компоненты закончились")



classIterableComponent(ABC):
    """Интерфейс для компонентов, которые можно обойти итератором"""


    @abstractmethod
    defcreate_iterator(self) -> ComponentIterator:
        pass
```

```
/root> mkdir asd
Введите дату создания папки asd: 12-12-12
asd добавлен в папочку root

/root> mkdir qwe
Введите дату создания папки qwe: 12-3-23
qwe добавлен в папочку root

/root> touch qew.txt
Введите дату создания файла qew.txt: 12-4-4
qew.txt добавлен в папочку root

/root> tree-random

Полное дерево файловой системы (СЛУЧАЙНЫЙ ПОРЯДОК):
 root/ (размер: 0 байт, создан: mkdir asd)
    [Итератор: компоненты перемешаны для случайного вывода]
   qwe/ (размер: 0 байт, создан: 12-3-23)
    [Итератор: компоненты перемешаны для случайного вывода]
   asd/ (размер: 0 байт, создан: 12-12-12)
    [Итератор: компоненты перемешаны для случайного вывода]
   qew.txt (размер: 0 байт, создан: 12-4-4, расширение: .txt)
```

Вывод: были отработаны базовые знания ООП языка программирования Python и паттернов проектирования при решении практических задач.